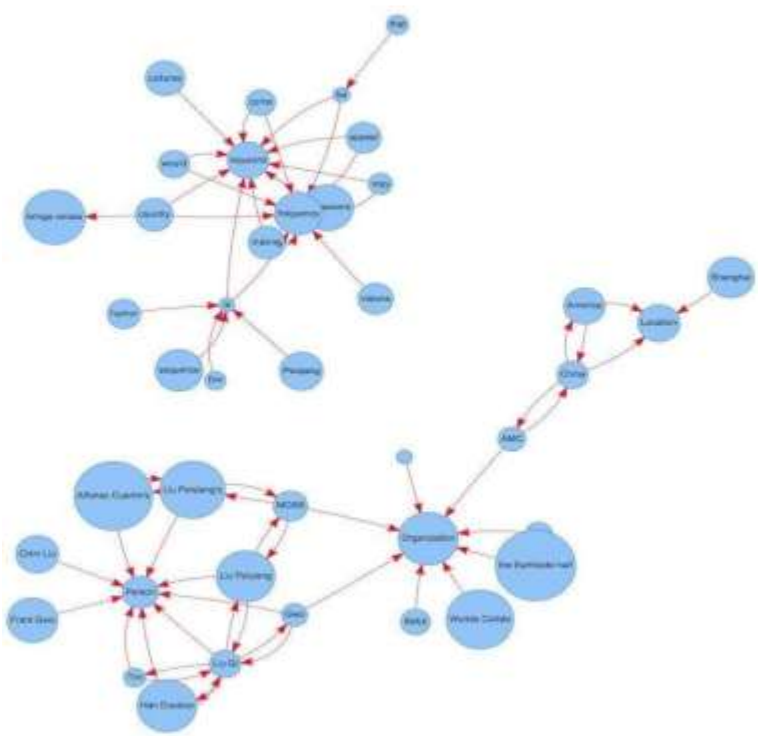
	Marwadi University **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No : 92301733009**

Aim: Representation of a document with their keywords using TextRank

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.

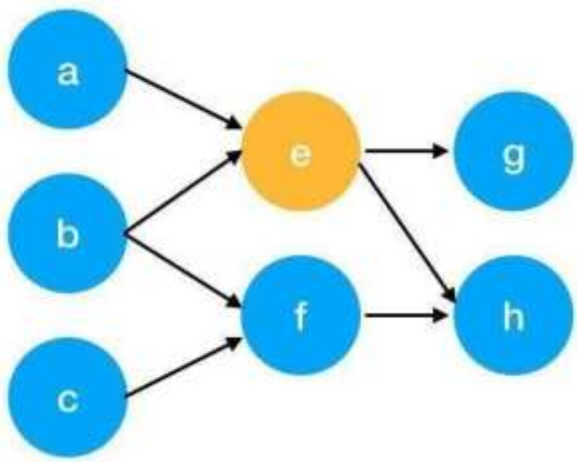


Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}, j \in \{a, b\}$$

$$\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) = \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b)$$

$$= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b)$$

$$= S(V_a) + \frac{1}{2} S(V_b)$$

We can get the weight of webpage e by the following function.

$$S(V_e) = (1-d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

Pre Lab Exercise:

1. What are the key components of TextRank?

- ✓ Graph Representation: Text is converted into a graph where words (or sentences) are nodes and relationships between them are edges.
- ✓ Co-occurrence Window: Words are connected if they appear within a fixed window size in the text.
- ✓ Edge Weights: The strength of connection between words is based on how frequently they co-occur.
- ✓ Ranking Algorithm (PageRank): An iterative algorithm assigns importance scores to nodes based on their connections.
- ✓ Convergence: The algorithm runs until the scores stabilize.
- ✓ Keyword Selection: Top-ranked words are selected as keywords.

2. What are some limitations of TextRank for keyword extraction?

- ✓ Ignores Semantic Meaning: It only considers word occurrence, not actual meaning or context.
- ✓ Sensitive to Window Size: Choosing an inappropriate window size can affect results.
- ✓ No Deep Language Understanding: Cannot capture synonyms or relationships like modern NLP models.
- ✓ May Extract Irrelevant Words: Sometimes common but less meaningful words may appear as keywords.
- ✓ Computational Cost: Iterative ranking can be slow for very large documents.

3. What are the advantages of TextRank for keyword extraction?

- ✓ Unsupervised Method: No training data is required.
- ✓ Language Independent: Can be applied to different languages easily.
- ✓ Simple and Effective: Easy to implement and works well for many tasks.
- ✓ Captures Word Importance: Uses graph structure to identify important words.
- ✓ Flexible: Can be used for both keyword extraction and text summarization.

Program (Code):

```
text= """I enjoy learning new things,
My favorite hobby is watching k-drama,
I am passionate about technology,
I love spending time with my myself,
I am always looking for ways to improve myself."""

#importing basic libraries
import nltk
import networkx as nx
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string

#downloading requirements

nltk.download('punkt')
nltk.download('stopwords')


def textrank_keyword(text, topn=5, window_size=4):
  words=word_tokenize(text.lower())
  stop_words=set(stopwords.words('english'))
  words=[w for w in words if w not in stop_words and w not in string.punctuation]
  graph=nx.Graph()

  for i, word in enumerate(words):
    for j in range(i+1, window_size+i):
      if (j<len(words)):
        graph.add_edge(word, words[j])

  scores=nx.pagerank(graph)
  ranked_words=sorted(scores.items(), key=lambda x:x[1], reverse=True)

  keywords=[word for word, score in ranked_words[:topn]]
  return keywords


nltk.download('punkt_tab')
```

```
print(textrank_keyword(text))
```

Results:

```
••• [nltk_data] Downloading package punkt to /root/nltk_data...
    [nltk_data]   Unzipping tokenizers/punkt.zip.
    [nltk_data] Downloading package stopwords to /root/nltk_data...
    [nltk_data]   Unzipping corpora/stopwords.zip.
    True
```

```
nltk.download('punkt_tab')

[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
print(textrank_keyword(text))

['things', 'always', 'favorite', 'time', 'hobby']
```

Observation:

- ✓ The TextRank algorithm was applied to extract keywords from the given document by converting the text into a graph structure.
- ✓ Words were treated as nodes and their co-occurrence relationships formed edges, allowing the algorithm to identify important terms.
- ✓ It was observed that frequently co-occurring and contextually important words received higher ranks.
- ✓ The algorithm successfully extracted meaningful keywords that represent the main idea of the document.
- ✓ However, some less important or common words were also included due to lack of semantic understanding.
- ✓ The results depended on parameters like window size and preprocessing steps (such as stop-word removal).

- ✓ The TextRank proved to be an effective unsupervised method for keyword extraction without requiring any

<table>
<tr><td rowspan="1">Marwadi University</td><td colspan="2">Marwadi University
Faculty of Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Artificial Intelligence (01CT0616)</td><td colspan="2">Aim: Representation of a document with their keywords using TextRank</td></tr>
<tr><td>Experiment No: 09</td><td>Date:</td><td>Enrolment No : 92301733009</td></tr>
</table>

training data.

Post Lab Exercise:

```
!pip install pytextrank
```

```
import spacy
import pytextrank
```

```
/usr/local/lib/python3.12/dist-packages
```

```
nlp = spacy.load("en_core_web_sm")
```

```
nlp.add_pipe("textrank")
```

```
<pytextrank.base.BaseTextRankFactory at 0x7a5b0f719cd0>
```

```
text = """
I am a student of Information and Communication Technology with a strong interest in Artificial Intelligence, Machine Learning, and Data Analytics. I am passionate about wor

I have hands-on experience in Python programming, data handling, and data visualization tools. Through academic projects and practical exposure, I have developed a solid und

I have also worked on projects involving web development, where I focused on creating user-friendly applications that provide valuable information and enhance user experienc

Additionally, I have explored hardware-based projects and participated in intership, which improved my creativity, teamwork, and ability to think innovatively under pressure

My key strengths include logical thinking, adaptability, and a continuous desire to learn new technologies. I am committed to building a successful career in AI/ML or Data A
"""
```

```
doc = nlp(text)
```

```
# Extract top 25 keywords
keywords = [phrase.text for phrase in doc._.phrases[:25]]
```

```
print("Top 25 Keywords:\n", keywords)
```

```
Top 25 Keywords:
 ['data handling', 'data visualization tools', 'data', 'academic projects', 'Data Analytics', 'projects', 'user experience', 'logical thinking', 'structured and semi-structur
```